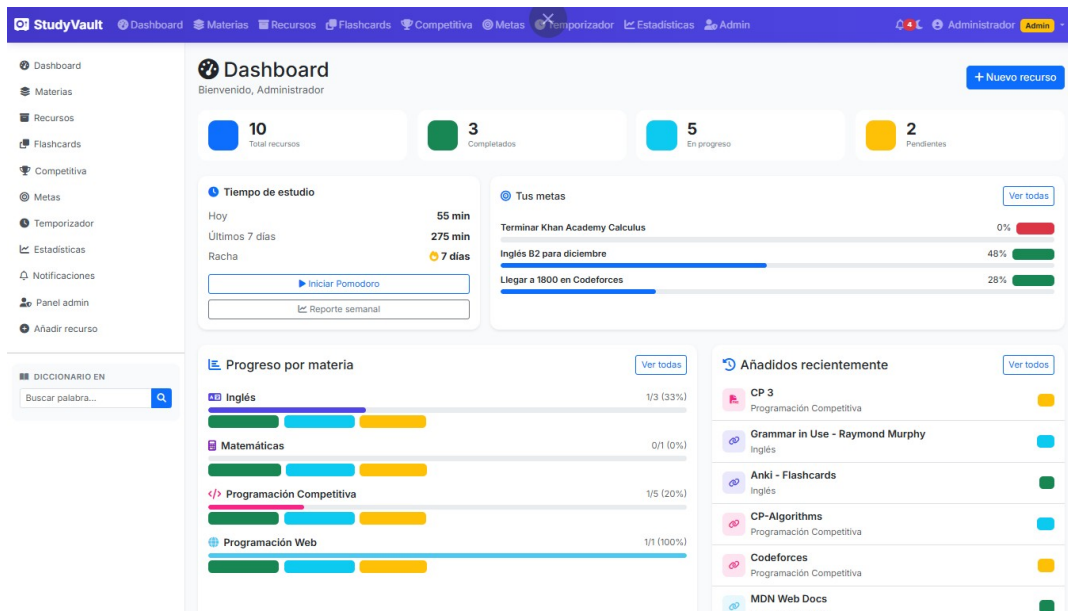


StudyVault

Gestor de material de estudio
con repetición espaciada



Documentación Formal del Proyecto

Autor: José Roberto García Correa
Materia: Programación Web
Semestre: Sexto
Stack: PHP 8 (OOP/MVC), MySQL, JavaScript, Bootstrap 5
Repositorio: StudyVault (rama develop)
Fecha: Mayo 2026

Documento entregable para evaluación final del proyecto.

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Planteamiento del problema	3
3.1. Contexto	3
3.2. Problema identificado	3
3.3. Propuesta de solución	4
3.4. Justificación	4
4. Requerimientos	4
4.1. Requerimientos funcionales	4
4.2. Requerimientos no funcionales	5
5. Diagrama de base de datos	6
5.1. Descripción de las tablas principales	8
6. Casos de uso	8
6.1. Actores	8
6.2. Diagrama de casos de uso	9
6.3. Descripción de los casos de uso principales	9
6.4. Otros casos de uso (resumen)	12
7. Capturas del sistema en funcionamiento	12
7.1. Pantalla principal y materias	12
7.2. Gestión de recursos	13
7.3. Flashcards y repetición espaciada	14
7.4. Diccionario integrado (4 APIs)	15
7.5. Programación competitiva (API Codeforces)	16
7.6. Metas y temporizador	16
7.7. Estadísticas (Chart.js)	17
7.8. Notificaciones, recuperación de contraseña y panel admin	18
8. Explicación técnica	20
8.1. Arquitectura general	20
8.2. Estructura de carpetas	20
8.3. Flujo de una petición	20
8.4. Algoritmo SM-2 (núcleo de retención)	21
8.5. Seguridad: defensa en profundidad	21
8.6. Integración con APIs externas	22
8.7. Pruebas automatizadas	22
8.8. Métricas finales	23
8.9. Conclusiones técnicas	23

1. Introducción

StudyVault es una aplicación web de gestión de estudio personal, diseñada para concentrar en un solo lugar el material de aprendizaje (libros, enlaces, problemas, vocabulario) y, sobre todo, medir el **avance real** del estudiante. A diferencia de los gestores de marcadores o las listas de tareas tradicionales, StudyVault aplica técnicas de eficacia probada como la **repetición espaciada (algoritmo SM-2)**, integra automáticamente el progreso del usuario con plataformas externas (**Codeforces**) mediante APIs públicas, y enriquece el aprendizaje del inglés con definiciones, colocaciones y frases reales obtenidas de tres servicios externos distintos.

El proyecto está construido siguiendo el patrón **Modelo-Vista-Controlador (MVC)** en **PHP orientado a objetos**, con persistencia en **MySQL** mediante **PDO con consultas preparadas**. La interfaz utiliza **Bootstrap 5**, **Chart.js** para gráficas, y **DataTables** para ordenamiento. La comunicación cliente-servidor se realiza por **AJAX/fetch**.

Esta documentación describe el problema que el proyecto resuelve, los objetivos planteados, los requerimientos cumplidos, el modelo de datos, los principales casos de uso y la explicación técnica de la arquitectura.

Audiencia del documento: docente evaluador del proyecto. Para una guía paso a paso de uso del sistema véase el *Manual de Usuario*; para el detalle del proceso de desarrollo véase el *Plan de Implementación*.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación web **full-stack** para la gestión personal del material de estudio que aplique técnicas de **retención basada en evidencia**, mida el **progreso real** del usuario, integre servicios externos vía **API REST**, y cumpla los principios de la **arquitectura MVC** con énfasis en **seguridad** (CSRF, anti-IDOR, hashing, throttling, CSP).

2.2 Objetivos específicos

- OE1.** Diseñar e implementar una **base de datos relacional** normalizada con 16 tablas, integridad referencial mediante claves foráneas y restricciones ON DELETE CASCADE, e índices en columnas de búsqueda.
- OE2.** Implementar un sistema de **autenticación segura** con `password_hash` (bcrypt), regeneración de sesión, cookies endurecidas (`HttpOnly`, `SameSite`, `Secure`) y **throttling** de intentos fallidos.
- OE3.** Definir **tres roles diferenciados** (admin, student, invitado) con privilegios y rutas protegidas.
- OE4.** Implementar el **algoritmo SM-2** (repetición espaciada) como función pura testeable, reutilizable para el repaso de flashcards y de problemas de programación competitiva.
- OE5.** Integrar **cuatro APIs públicas** (Free Dictionary, Datamuse, Tatoeba, Codeforces) mediante **proxies PHP** con caché y **rate limiting**, evitando CORS y exposición de límites de API.
- OE6.** Implementar **funcionalidades dinámicas** de UI (búsqueda en vivo, filtros, paginación server-side, ordenamiento client-side con DataTables, CRUD AJAX, subida de archivos con validación MIME).

- OE7.** Generar **6 gráficas estadísticas** con **Chart.js** a partir de datos agregados en servidor (minutos por día, tarjetas por estado SM-2, distribución CEFR, problemas CP por tag, curva de rating, histograma de dificultad).
- OE8.** Implementar un **centro de notificaciones** in-app con reglas idempotentes diarias (repasos pendientes, metas atrasadas, racha en riesgo) e integración con la **Notification API** del navegador.
- OE9.** Aplicar técnicas de **seguridad defensiva**: tokens CSRF en todo POST, consultas PDO preparadas, anti-IDOR, cabeceras de seguridad (**X-Frame-Options**, **X-Content-Type-Options**, **Referrer-Policy**, **CSP**), **.htaccess** anti-ejecución en directorio de uploads.
- OE10.** Producir documentación completa: README, Manual de Usuario, Plan de Implementación con bitácora, Guion de Exposición y este documento formal.

3. Planteamiento del problema

3.1 Contexto

Los estudiantes autodirigidos – aquellos que aprenden por su cuenta materias como idiomas extranjeros, ciencias o programación competitiva – acumulan material de estudio disperso: libros en PDF, cursos en video, problemas pendientes de resolver, vocabulario por memorizar, anotaciones rápidas. Para administrar este material recurren típicamente a varias herramientas separadas:

- **Anki** para memorización de vocabulario.
- **Codeforces** (u otras plataformas) para programación competitiva.
- **Google Drive** u **OneDrive** para alojar PDFs.
- **Notion** u **Obsidian** para apuntes.
- **Hojas de cálculo** para llevar registro manual de avance.

3.2 Problema identificado

Esta fragmentación produce **cuatro problemas concretos**:

1. **Sin visión unificada del avance.** El estudiante no sabe cuánto tiempo ha estudiado en total, cuánto vocabulario ya domina, ni cuántos problemas ha resuelto. Cada herramienta da su propia métrica parcial.
2. **Marcar “hecho” no equivale a aprender.** Una lista de tareas no mide retención. Un libro marcado como terminado puede haberse olvidado por completo en dos semanas.
3. **La medición del progreso es subjetiva y manual.** Sin integración automática, el estudiante anota a mano lo que ha estudiado, lo que es propenso a olvido o autoengaño.
4. **El estudio del vocabulario fuera de contexto es ineficaz.** Aprender palabras como pares “palabra-traducción” no enseña a usarlas; falta el contexto de *colocaciones* (con qué palabras se usa) y *ejemplos reales*.

3.3 Propuesta de solución

StudyVault aborda los cuatro problemas anteriores en una única aplicación:

1. **Visión unificada:** un dashboard que agrega tiempo de estudio, racha, metas con % ponderado, recursos recientes y tarjetas por repasar.
2. **Mide retención:** algoritmo **SM-2** sobre flashcards y sobre problemas de CP. El usuario no decide qué repasar; el sistema le presenta sólo lo que toca hoy.
3. **Automatización:** integración con la **API pública de Codeforces** para importar rating y problemas resueltos sin captura manual.
4. **Aprendizaje en contexto:** cada palabra guardada desde el diccionario incluye su *definición* (Free Dictionary), *colocaciones* (Datamuse) y *frases reales* (Tatoeba).

3.4 Justificación

El proyecto cumple un doble objetivo:

- **Académico:** demostrar la aplicación de todos los conceptos del curso de Programación Web (HTML/CSS, JavaScript, PHP OOP, MySQL/PDO, MVC, AJAX, consumo de APIs, sesiones y seguridad) en un proyecto integrador con utilidad real.
- **Personal y profesional:** construir una herramienta que el propio autor utilizará para preparar exámenes oficiales de inglés (CEFR B2) y para subir su rating competitivo en Codeforces.

4. Requerimientos

4.1 Requerimientos funcionales

ID	Descripción
RF-01	El sistema permite el registro de usuarios con nombre, correo único y contraseña (mínimo 8 caracteres).
RF-02	El sistema permite el inicio y cierre de sesión, con bloqueo tras 5 intentos fallidos en 15 minutos.
RF-03	El sistema permite a un usuario sin sesión (rol Invitado) explorar las plantillas públicas de metas.
RF-04	El usuario administrador puede listar todos los usuarios, buscarlos por nombre o correo, y activar/desactivar cuentas.
RF-05	El usuario puede crear, editar y eliminar materias propias (con nombre, descripción, ícono y color).
RF-06	El usuario puede crear, editar y eliminar recursos (enlace, PDF, nota o video) asociados a una materia.
RF-07	El usuario puede subir archivos PDF o imagen (máx. 50 MB) con validación de tipo MIME.
RF-08	El usuario puede buscar y filtrar recursos en tiempo real por título, materia, tipo o estado.
RF-09	El usuario puede ordenar las tablas (Recursos, Flashcards, Competitiva) haciendo clic en los encabezados.
RF-10	El usuario puede dividir un recurso en unidades (capítulos/lecciones) y ver el % real de avance.

ID	Descripción
RF-11	El usuario puede crear flashcards manualmente o desde el diccionario, con anverso, reverso, ejemplo y nivel CEFR.
RF-12	El sistema implementa el algoritmo SM-2 para programar el repaso de cada flashcard.
RF-13	El usuario puede estudiar tarjetas en tres modos: Clásico, Cloze y Producción.
RF-14	El usuario puede exportar e importar tarjetas en formato CSV compatible con Anki.
RF-15	El sistema consulta la API de Free Dictionary para obtener definición, fonética y audio de una palabra.
RF-16	El sistema consulta la API de Datamuse para obtener colocaciones (palabras que coexisten).
RF-17	El sistema consulta la API de Tatoeba para obtener frases de ejemplo reales.
RF-18	El usuario vincula su handle de Codeforces y sincroniza su rating y problemas resueltos.
RF-19	El usuario puede programar el repaso espaciado (SM-2) de problemas de CP.
RF-20	El sistema sugiere problemas de CP no resueltos en el rango $\text{rating}+100 \dots +300$ (zona $i+1$).
RF-21	El usuario puede crear metas con título, descripción y fecha objetivo.
RF-22	El usuario asocia recursos a metas con un peso, y el sistema calcula el % ponderado.
RF-23	El sistema calcula el ritmo (“En camino” / “Atrasado”) comparando el % real vs el % esperado.
RF-24	El usuario puede publicar una meta como plantilla pública y otros usuarios pueden clonarla.
RF-25	El usuario puede iniciar un Pomodoro de 25/5 que registra los minutos reales al completarse.
RF-26	El sistema muestra minutos de hoy, semana, racha de días y heatmap de 84 días.
RF-27	El sistema presenta 6 gráficas Chart.js con datos reales del usuario en la página de Estadísticas.
RF-28	El sistema genera notificaciones automáticas idempotentes (1 por día por tipo) ante: repaos pendientes, metas atrasadas, racha en riesgo, meta diaria sin cubrir.
RF-29	El usuario puede optar por avisos del navegador mediante la Notification API.
RF-30	El usuario puede solicitar la recuperación de contraseña vía un token de expiración 1 hora.
RF-31	El usuario puede exportar el Reporte semanal y una Meta a PDF mediante <code>window.print()</code> .
RF-32	El sistema registra cada acción importante del usuario en una bitácora (<code>activity_log</code>).

4.2 Requerimientos no funcionales

ID	Descripción
RNF-01	Seguridad — anti SQL Injection: todas las consultas usan PDO con <i>prepared statements</i> ; nunca se concatena entrada de usuario en SQL.
RNF-02	Seguridad — anti CSRF: todo POST requiere un token criptográfico por sesión (32 bytes hex, comparación con <code>hash_equals</code>).
RNF-03	Seguridad — anti IDOR: toda consulta filtra por <code>user_id</code> ; existen pruebas automatizadas (<code>tests/security_test.php</code>) que verifican el aislamiento entre usuarios.
RNF-04	Seguridad — anti XSS: toda salida pasa por <code>htmlspecialchars()</code> ; CSP restringe orígenes de scripts y estilos.
RNF-05	Seguridad — contraseñas: <code>password_hash(\$pwd, PASSWORD_DEFAULT)</code> con <code>bcrypt</code> ; nunca se almacena plaintext.
RNF-06	Seguridad — sesiones: <code>session_regenerate_id(true)</code> al autenticar; cookies con <code>HttpOnly</code> , <code>SameSite=Lax</code> , y <code>Secure</code> bajo HTTPS.
RNF-07	Seguridad — cabeceras: <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: SAMEORIGIN</code> , <code>Referrer-Policy: strict-origin-when-cross-origin</code> , Content-Security-Policy restrictiva.
RNF-08	Seguridad — uploads: <code>.htaccess</code> en <code>assets/uploads/</code> desactiva la ejecución de PHP en ese directorio.
RNF-09	Seguridad — credenciales: configuración en <code>.env</code> fuera del control de versiones.
RNF-10	Rendimiento: índices en columnas críticas (<code>user_id</code> , <code>due_date</code> , <code>status</code> , <code>cefr_level</code> , <code>cf_rating</code>).
RNF-11	Rendimiento: caché local de definiciones del diccionario (30 días) y del <code>problemset</code> de Codeforces.
RNF-12	Rendimiento: <i>rate limiting</i> por sesión sobre los proxies a APIs externas.
RNF-13	Mantenibilidad: arquitectura MVC clara; clases con responsabilidad única; autoload PSR-0 simple.
RNF-14	Mantenibilidad: cambios documentados en una bitácora (<code>PLAN_IMPLEMENTACION.md §G</code>) con archivos modificados y cómo revertir.
RNF-15	Verificabilidad: 53 <i>assertions</i> en <code>tests/</code> sobre lógica pura crítica (SM-2, progreso ponderado, paginación, throttle de login, IDOR).
RNF-16	Usabilidad: interfaz responsive con Bootstrap 5; modo oscuro persistente; atajos de teclado en el estudio de flashcards.
RNF-17	Compatibilidad: funciona en Chrome, Firefox y Edge actuales; PHP 8+ y MySQL 5.7+ / MariaDB 10.2+.
RNF-18	Internacionalización: interfaz en español; UTF-8 (<code>utf8mb4</code>) en BD para soportar acentos y emojis.
RNF-19	Robustez: manejo de errores centralizado con <code>set_exception_handler</code> ; los fallos de las APIs externas devuelven JSON controlado, no errores 500.

5. Diagrama de base de datos

El modelo relacional consta de **16 tablas** agrupadas en seis categorías. Todas las claves foráneas usan `ON DELETE CASCADE` para mantener integridad referencial. Las relaciones uno-a-muchos se identifican con 1:N; la única N:M se materializa en la tabla intermedia `goal_resources`.

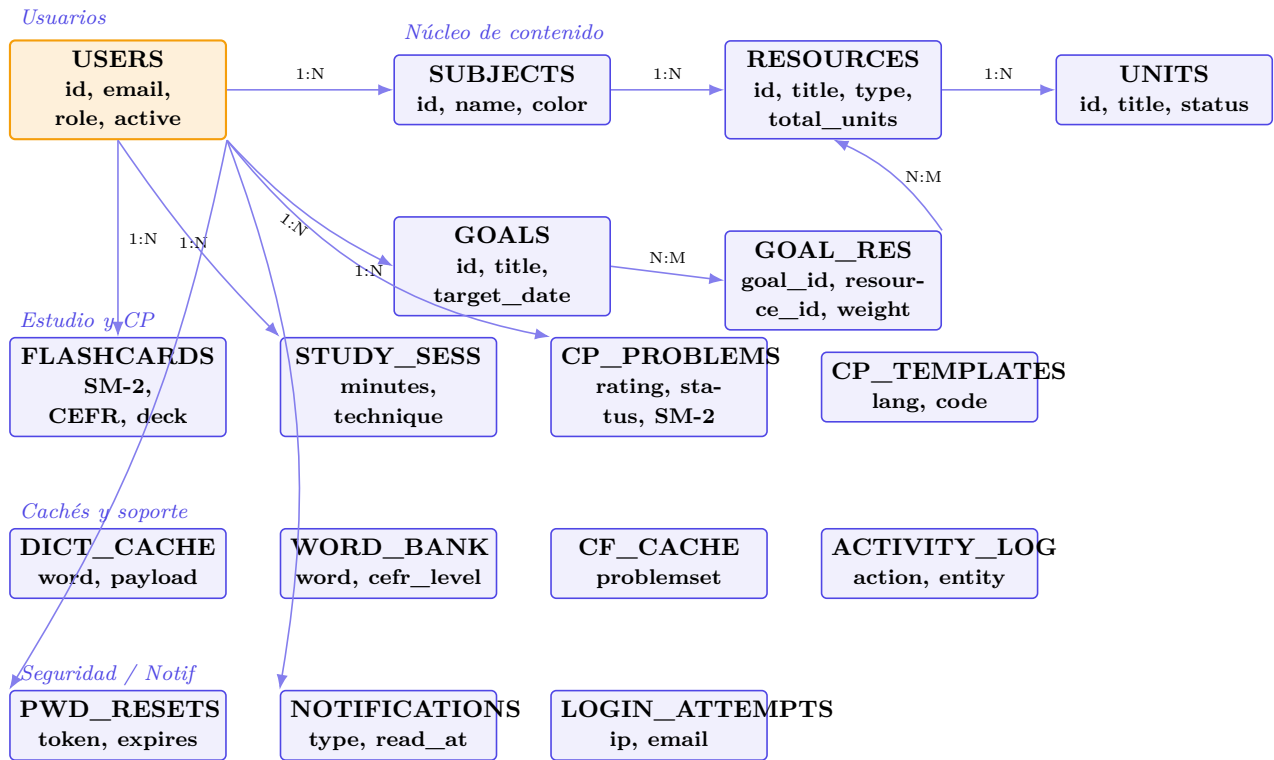


Figura 1: Diagrama relacional simplificado: 16 tablas agrupadas en 5 capas. *USERS* es la entidad pivote; todas las tablas de dominio se relacionan con ella mediante FK *user_id*.

5.1 Descripción de las tablas principales

Tabla	Propósito
<code>users</code>	Cuentas de usuario con <code>role</code> (admin/student), <code>active</code> , <code>cf_handle</code> , <code>daily_goal_minutes</code> .
<code>subjects</code>	Materias creadas por cada usuario, con ícono y color.
<code>resources</code>	Material de estudio (enlace, PDF, nota o video) asociado a una materia.
<code>units</code>	Capítulos / lecciones dentro de un recurso (resuelve “el libro grande”).
<code>goals</code>	Metas de estudio con fecha objetivo y estado (active/paused/done).
<code>goal_resources</code>	Relación N:M entre metas y recursos con <code>weight</code> para progreso ponderado.
<code>flashcards</code>	Tarjetas SM-2 (genéricas: <code>deck</code> = vocab / cp), con <code>ease_factor</code> , <code>interval_days</code> , <code>due_date</code> .
<code>study_sessions</code>	Bloques de estudio (Pomodoro, lectura), con minutos reales registrados.
<code>cp_problems</code>	Problemas de Codeforces sincronizados, con estado, rating, tags y campos SM-2.
<code>cp_templates</code>	Plantillas de código del usuario (DSU, BFS, etc.).
<code>cf_problemset_cache</code>	Caché del catálogo completo de Codeforces para sugerencias.
<code>dictionary_cache</code>	Respuestas cacheadas de Free Dictionary (30 días).
<code>word_bank</code> , <code>word_enrichment</code>	Tablas auxiliares para futuro banco CEFR completo.
<code>activity_log</code>	Bitácora de acciones del usuario.
<code>login_attempts</code>	Registro de intentos de login (anti-bruteforce).
<code>password_resets</code>	Tokens de recuperación con expiración.
<code>notifications</code>	Centro in-app de notificaciones generadas por reglas idempotentes.

6. Casos de uso

6.1 Actores

- **Invitado** – visitante sin cuenta. Acceso de solo lectura al catálogo público de plantillas.
- **Estudiante (`student`)** – usuario autenticado regular. Gestiona su propio material y estudia.
- **Administrador (`admin`)** – todo lo del estudiante más gestión de usuarios y métricas globales.
- **Sistema** – realiza tareas automáticas (generar notificaciones, calcular pace, ejecutar SM-2).
- **Codeforces / Free Dictionary / Datamuse / Tatoeba** – sistemas externos consumidos vía API.

6.2 Diagrama de casos de uso

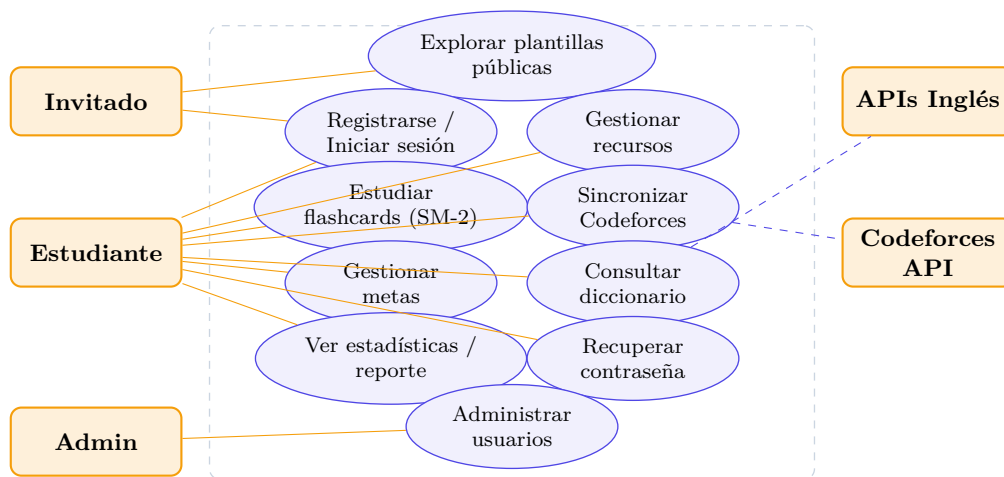


Figura 2: Diagrama UML simplificado de casos de uso. Línea continua naranja: actor humano → caso de uso. Línea punteada índigo: comunicación con sistema externo.

6.3 Descripción de los casos de uso principales

CU-01: Estudiar flashcards (algoritmo SM-2)

Actor:	Estudiante
Pre-cond:	Sesión iniciada; existen flashcards con <code>due_date ≤ hoy</code> .
Flujo:	1. El estudiante entra al menú Flashcards y pulsa <i>Estudiar</i>. 2. El sistema obtiene la cola: <code>SELECT * FROM flashcards WHERE user_id=? AND due_date ≤ CURRENT_DATE</code>. 3. El sistema muestra la primera tarjeta (anverso oculto al usuario). 4. El estudiante pulsa Espacio para voltearla. 5. El estudiante califica con 1–4 (Otra vez / Difícil / Bien / Fácil). 6. El sistema invoca <code>Flashcard::sm2(\$ef, \$reps, \$interval, \$quality)</code> y persiste el nuevo <code>due_date</code>, <code>ease_factor</code> y <code>status</code>. 7. Vuelve al paso 3 hasta agotar la cola.
Post-cond:	Las tarjetas calificadas tienen un nuevo <code>due_date</code> futuro acorde al algoritmo SM-2.
Excepción:	Si no hay tarjetas vencidas, se muestra un mensaje “no tienes nada que repasar hoy”.

CU-02: Sincronizar Codeforces

Actor:	Estudiante (consume API de Codeforces).
Pre-cond:	El usuario ha guardado su <code>cf_handle</code> .
Flujo:	1. El estudiante pulsa <i>Sincronizar</i> en la pantalla de Competitiva. 2. El sistema invoca <code>user.info?handles=<handle></code> y guarda el <code>rating</code>. 3. El sistema invoca <code>user.status?handle=<handle>&count=3000</code> y filtra los envíos con verdict OK. 4. Para cada problema resuelto: <code>INSERT INTO cp_problems ... ON DUPLICATE KEY UPDATE</code> para evitar duplicados. 5. El sistema actualiza <code>users.cf_rating</code> y <code>users.cf_synced_at</code>. 6. Se muestra una alerta con el número de problemas importados/actualizados.
Post-cond:	La tabla <code>cp_problems</code> contiene los problemas resueltos del usuario; el dashboard CP refleja el nuevo rating.
Excepción:	Si la API de Codeforces no responde, se muestra un mensaje controlado sin error 500.

CU-03: Consultar diccionario y guardar tarjeta

Actor:	Estudiante (consume Free Dictionary, Datamuse y Tatoeba).
Pre-cond:	Sesión iniciada.
Flujo:	1. El estudiante escribe una palabra inglesa en el widget de la barra lateral. 2. El sistema busca primero en <code>dictionary_cache</code>. Si no está, llama a Free Dictionary y guarda en caché. 3. Paralelamente llama a Datamuse (colocaciones) y a Tatoeba (frases). 4. Renderiza definición, fonética, audio, “se usa con” y ejemplos. 5. El estudiante pulsa <i>Guardar como tarjeta</i>. 6. El sistema inserta una fila en <code>flashcards</code> con <code>deck='vocab'</code>.
Post-cond:	Existe una nueva flashcard vencida para hoy.

CU-04: Gestionar usuarios (panel administrativo)

Actor:	Administrador.
Pre-cond:	Sesión con <code>role='admin'</code> ; el sistema ha validado <code>requireAdmin()</code> .
Flujo:	1. El admin entra al menú Admin. 2. El sistema muestra 8 tarjetas con totales globales (usuarios, materias, recursos, etc.). 3. Muestra el listado paginado de usuarios con búsqueda por nombre/correo. 4. El admin pulsa <i>Desactivar</i> en un usuario; el sistema verifica vía CSRF y actualiza <code>users.active=0</code>. 5. El usuario desactivado no podrá iniciar sesión (mensaje claro al intento).
Post-cond:	El usuario seleccionado queda inactivo; queda registro en <code>activity_log</code> .

CU-05: Recuperar contraseña

Actor:	Cualquier usuario que olvidó su contraseña.
Flujo:	1. El usuario va a la pantalla de login y pulsa <i>¿Olvidaste tu contraseña?</i>. 2. Escribe su correo registrado. 3. El sistema invalida cualquier token anterior (<code>used=1</code>) y genera uno nuevo: <code>bin2hex(random_bytes(32))</code> con expiración a 1h. 4. El sistema muestra el enlace en pantalla (modo demo sin SMTP) o lo envía por correo (producción). 5. El usuario sigue el enlace, define nueva contraseña (≥ 8 caracteres), confirma. 6. El sistema actualiza <code>users.password</code> con <code>password_hash()</code> y marca el token como usado.
Post-cond:	La contraseña ha sido renovada; el usuario puede iniciar sesión con ella.

6.4 Otros casos de uso (resumen)

ID	Caso de uso	Actor / Notas
CU-06	Crear y administrar materias y recursos	Estudiante (CRUD vía AJAX)
CU-07	Subir un libro en PDF (hasta 50 MB)	Estudiante (validación MIME)
CU-08	Dividir un libro en unidades / capítulos	Estudiante (generación masiva)
CU-09	Crear meta con fecha y agregar recursos ponderados	Estudiante
CU-10	Iniciar Pomodoro y registrar tiempo de estudio	Estudiante
CU-11	Ver gráficas de progreso (Chart.js)	Estudiante
CU-12	Recibir y leer notificaciones in-app	Estudiante
CU-13	Activar Notification API del navegador	Estudiante (opt-in)
CU-14	Publicar meta como plantilla pública	Estudiante
CU-15	Clonar plantilla pública a la cuenta propia	Estudiante
CU-16	Explorar plantillas públicas sin cuenta	Invitado
CU-17	Exportar reporte semanal o meta a PDF	Estudiante
CU-18	Exportar/importar tarjetas en CSV (Anki)	Estudiante
CU-19	Cambiar a modo oscuro	Cualquier usuario

7. Capturas del sistema en funcionamiento

7.1 Pantalla principal y materias

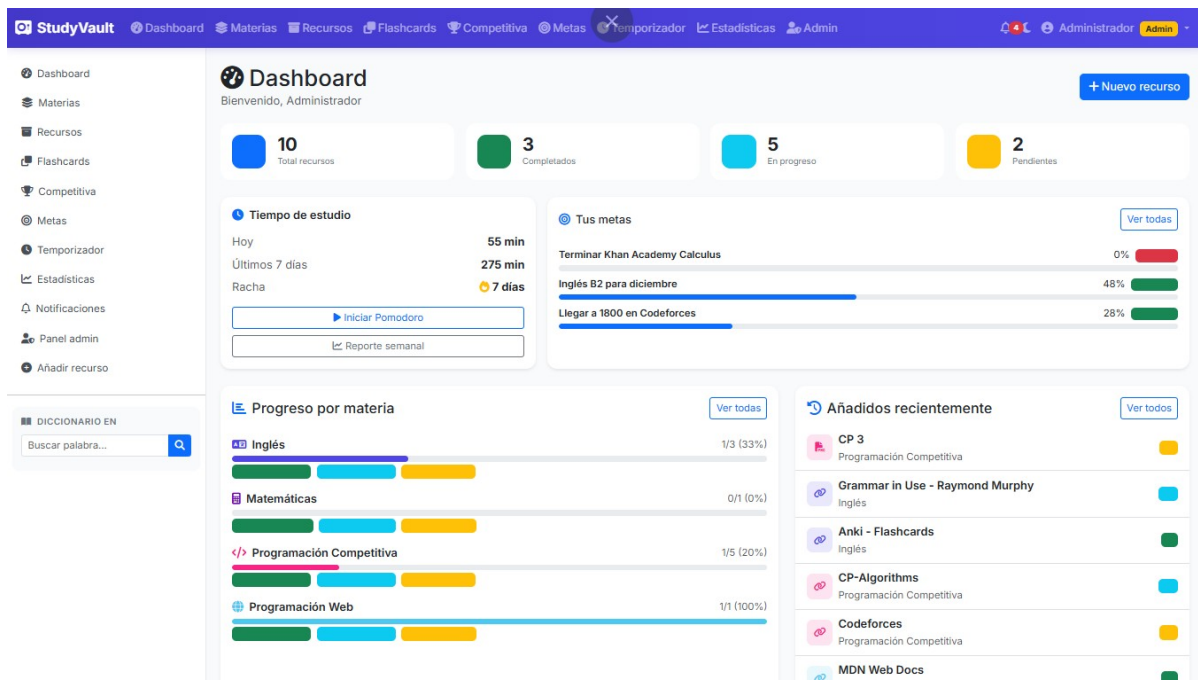


Figura 3: **Dashboard:** resumen tras iniciar sesión con cuenta administradora. Muestra tarjetas de stats, tiempo de estudio (hoy/semana), racha, metas en curso con su % y pace, y recursos recientes.

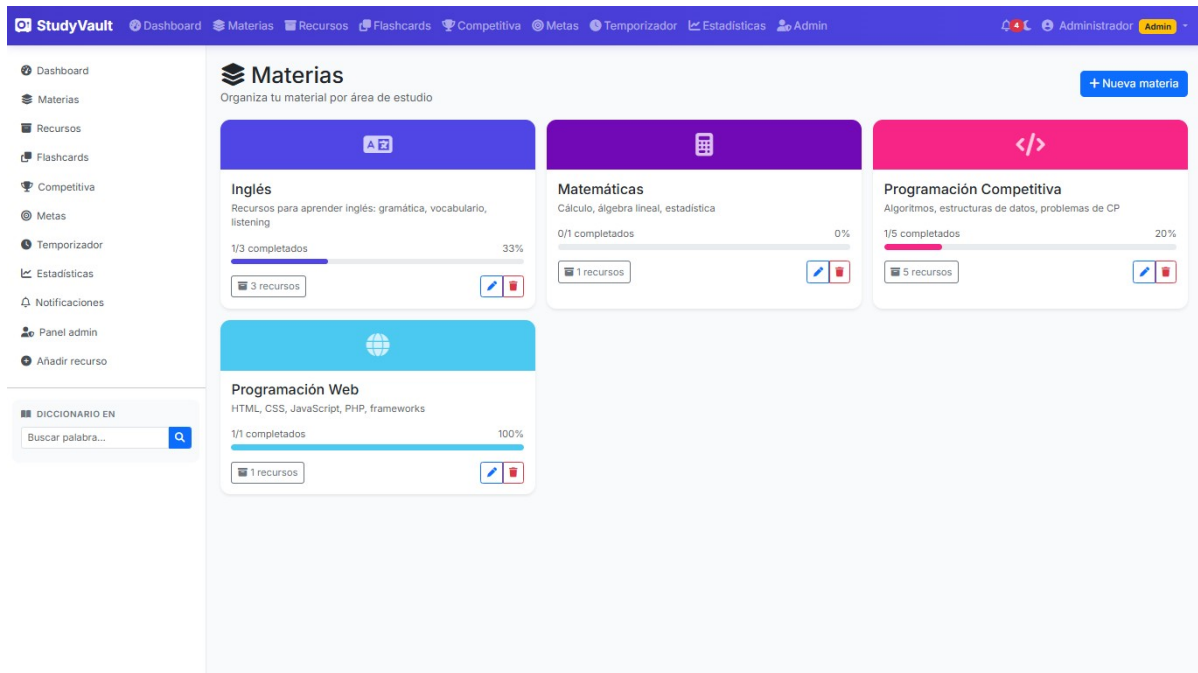


Figura 4: **Materias**: listado de áreas temáticas con ícono, color y % de avance individual.

7.2 Gestión de recursos

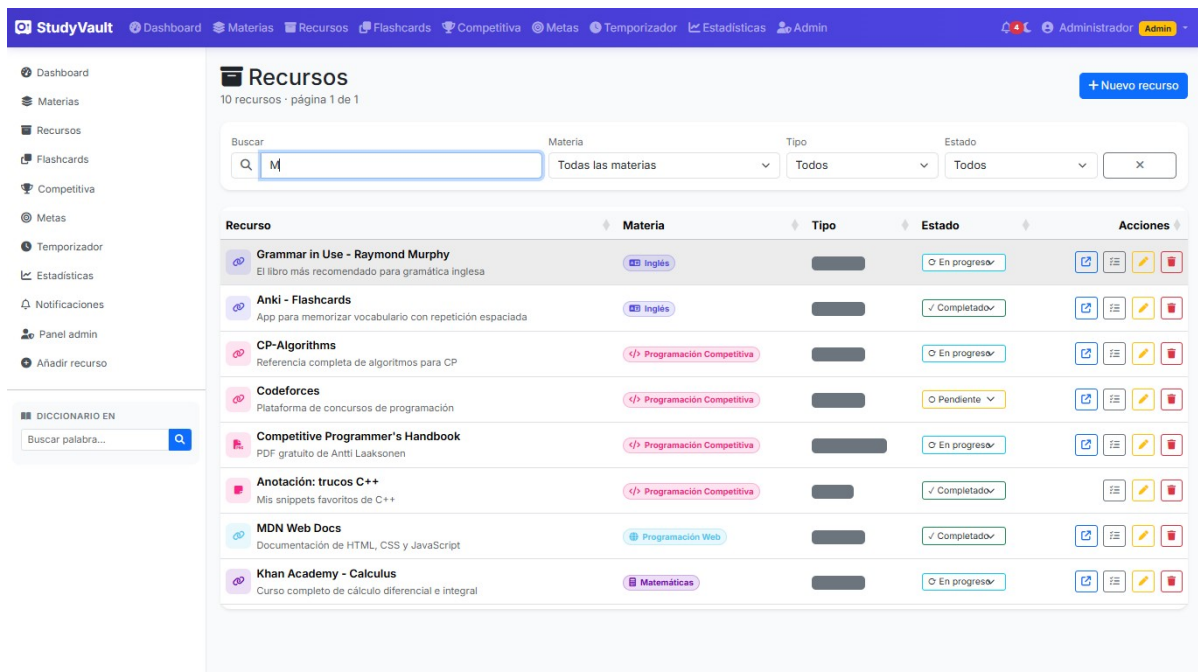


Figura 5: **Recursos**: tabla con búsqueda en vivo (AJAX), filtros por materia / tipo / estado, paginación server-side y ordenamiento client-side por columna (DataTables).

Nuevo recurso

Título *
ej. CP-Algorithms - Referencia de algoritmos

Descripción
Breve descripción del recurso...

Tipo *
PDF / Archivo

Materia *
Selecciona una materia

Archivo (PDF o imagen)
Seleccionar archivo Sin archivos seleccionados
Máximo 50 MB. Formatos: PDF, JPG, PNG, GIF, WEBP

Estado
Pendiente

Guardar recurso Cancelar

Figura 6: **Nuevo recurso**: formulario con validación MIME y soporte para subir archivos PDF / imagen hasta 50 MB.

7.3 Flashcards y repetición espaciada

Flashcards
Repaso con repetición espaciada (SM-2)

Estudiar Exportar Importar Por nivel Nueva tarjeta

9 Por repasar hoy 20 Total 8 Aprendiendo 5 Maduras 1 Dominadas

Mostrar 19

Anverso	Reverso	Mazo	Estado	Próx. repaso	Acciones
book	A collection of sheets of paper bound together to hinge at o...			2028-05-29	
overwhelmed	abrumado			2028-05-29	
ubiquitous	omnipresente			2028-05-29	
hello	hola			2028-05-29	
embark	embarcarse			2028-05-29	
lunch	almuerzo			2028-05-29	
¿Qué es un DSU?	Disjoint Set Union (Union-Find) con path compression y union...			2028-05-29	
¿Para qué sirve Dijkstra?	Camino mínimo desde un origen en grafo con pesos no negativo...			2028-05-29	
¿Qué es memoización?	Cachear resultados de subproblemas en DP top-down.			2028-05-29	
although	sinónimo: even though			2028-05-30	

1-10 de 20

Figura 7: **Flashcards**: listado con estadísticas por estado del algoritmo SM-2 (nuevas, aprendiendo, maduras, dominadas).

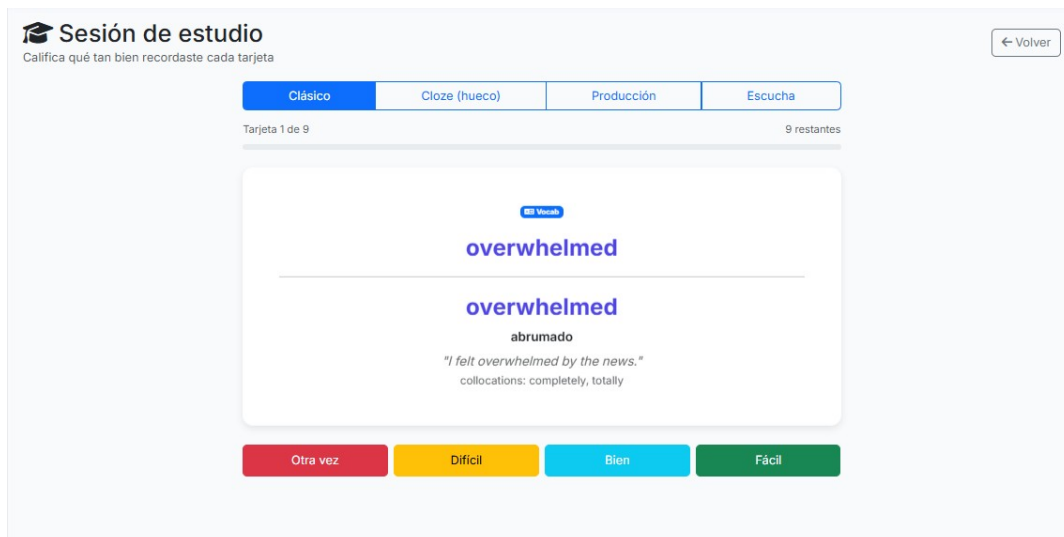


Figura 8: **Sesión de estudio:** tarjeta volteable con los 4 botones de calificación SM-2 + atajos de teclado.

7.4 Diccionario integrado (4 APIs)

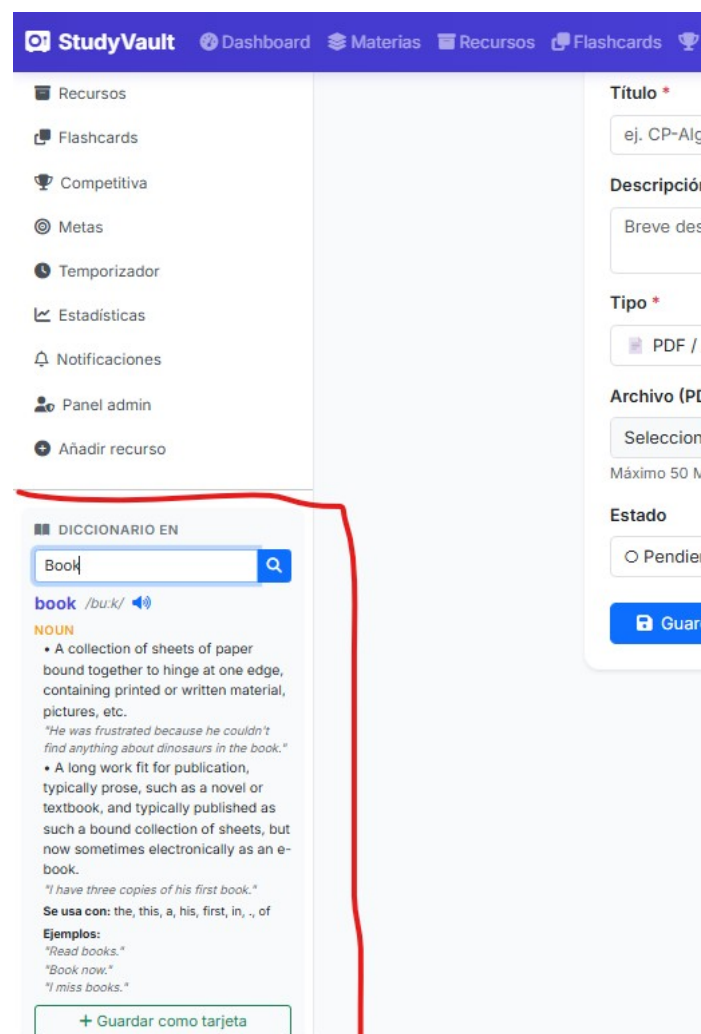


Figura 9: **Diccionario inglés:** barra lateral consumiendo Free Dictionary (definición + audio), Datamuse (colocaciones) y Tatoeba (frases reales), con botón para guardar la palabra como flashcard.

7.5 Programación competitiva (API Codeforces)

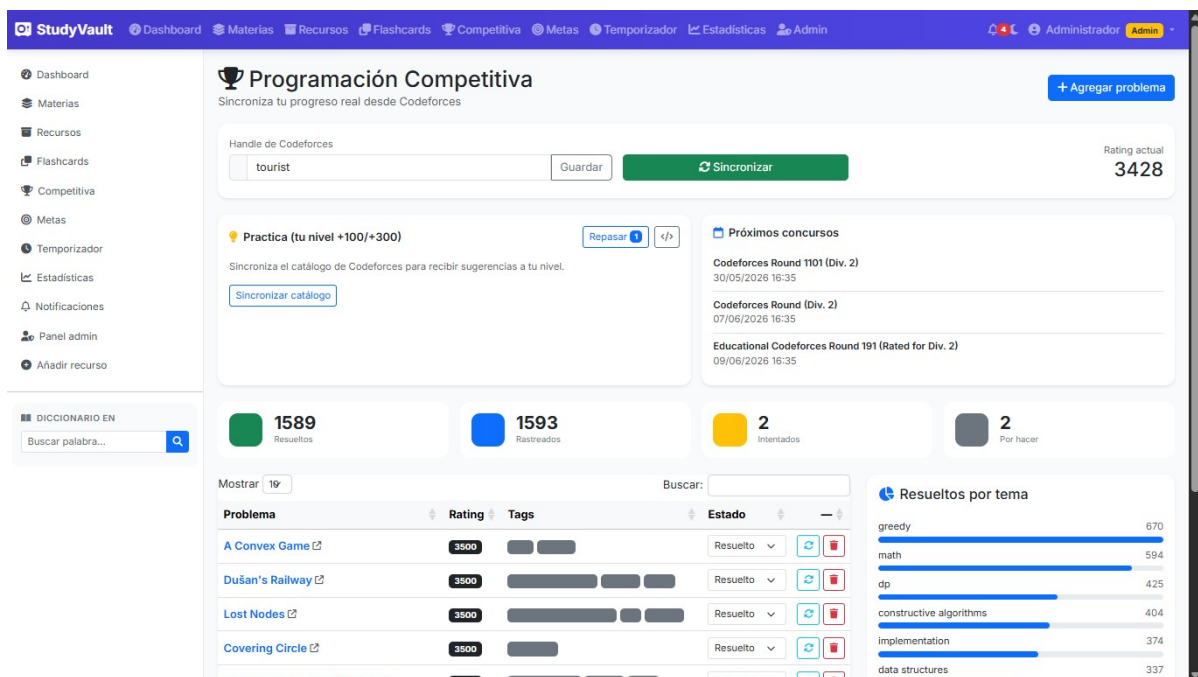


Figura 10: **Competitiva:** handle sincronizado, rating actual, sugerencias de problemas en el rango $i+1$, tabla de problemas resueltos y análisis por tag (debilidades).

7.6 Metas y temporizador

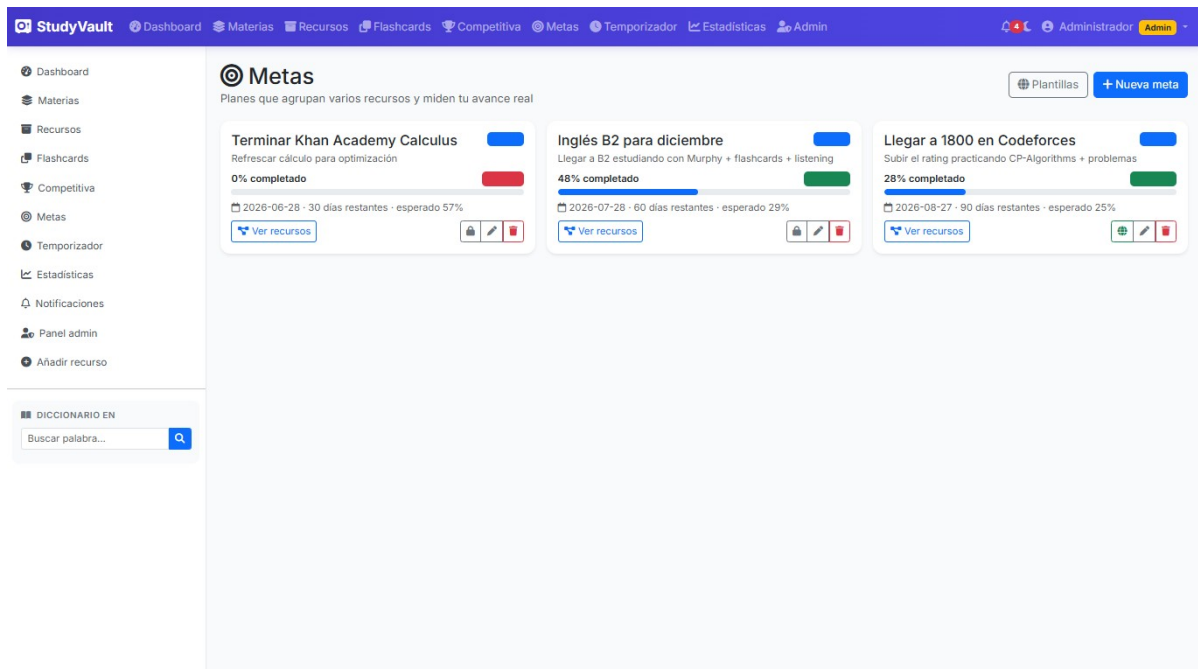


Figura 11: **Metas:** progreso ponderado por recurso, fecha objetivo, etiqueta de pace (“En camino” / “Atrasado”) y opción de publicar como plantilla pública.

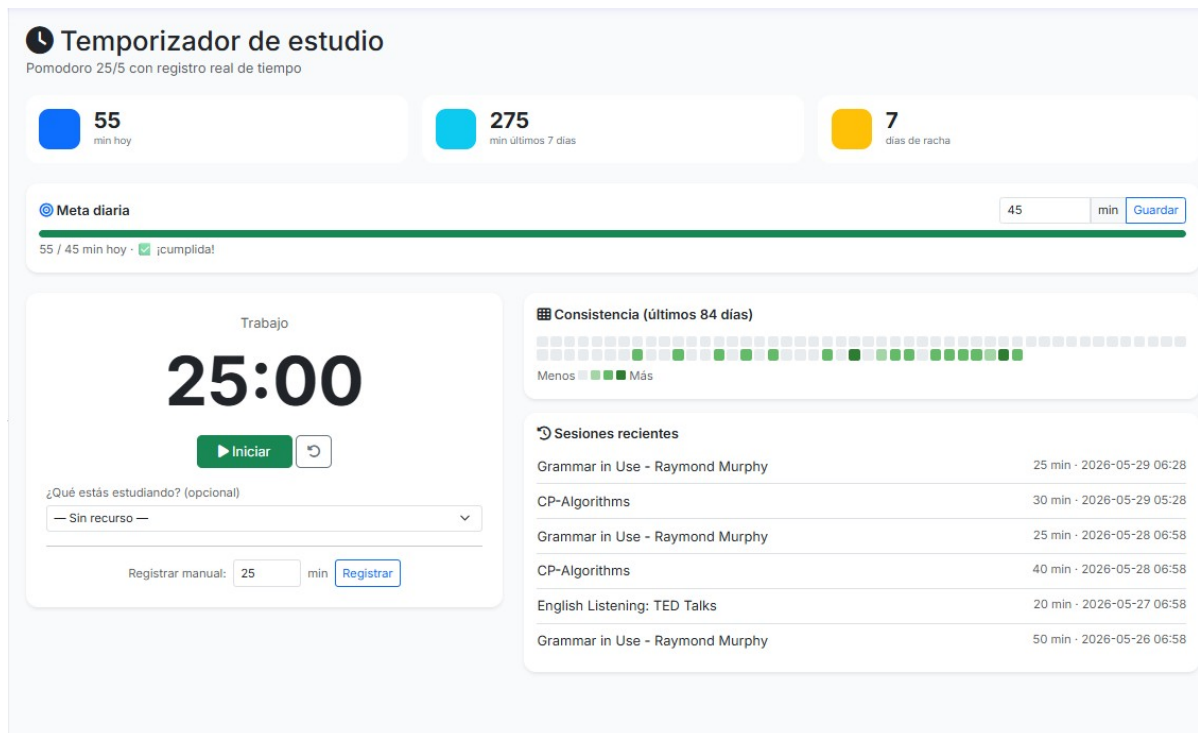


Figura 12: **Temporizador Pomodoro 25/5**: minutos de hoy y semana, racha de días consecutivos, meta diaria configurable y heatmap de 84 días.

7.7 Estadísticas (Chart.js)

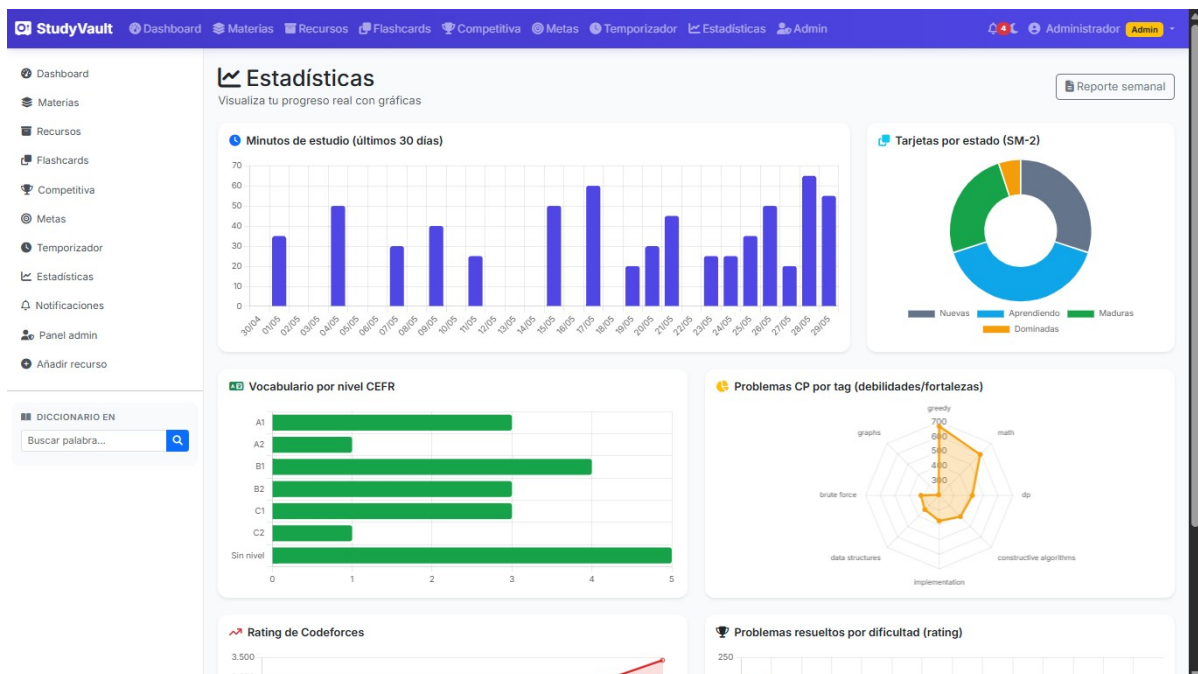


Figura 13: **Estadísticas**: 6 gráficas Chart.js con datos reales – minutos por día (30d), tarjetas por estado SM-2, vocabulario CEFR, CP por tag (radar), curva de rating CF e histograma de dificultad.

7.8 Notificaciones, recuperación de contraseña y panel admin

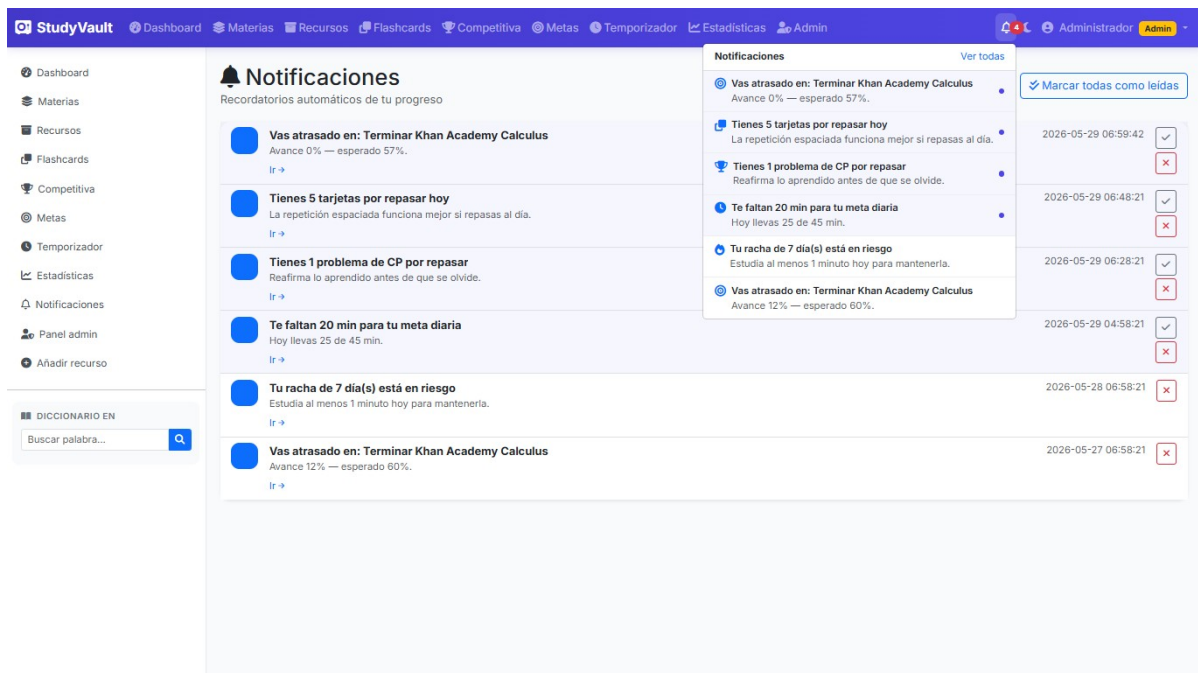


Figura 14: **Centro de notificaciones:** badge en navbar + dropdown + página completa con avisos generados automáticamente por las 5 reglas idempotentes diarias.

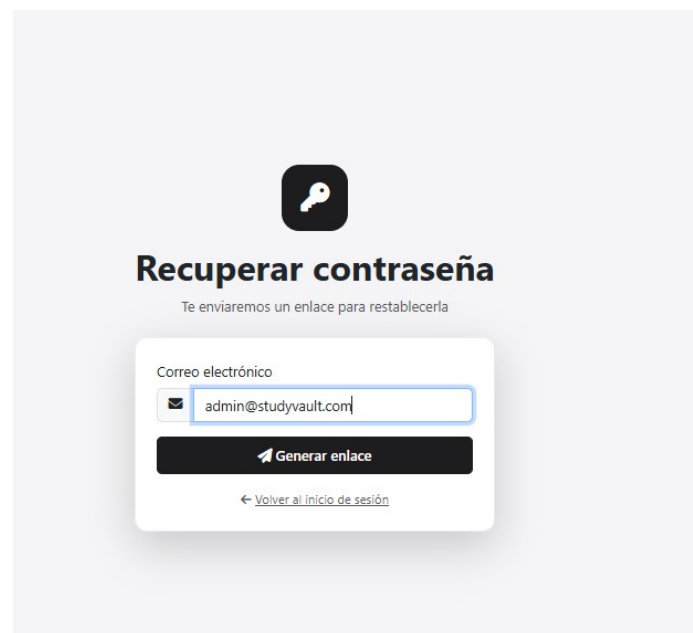


Figura 15: **Recuperación de contraseña:** formulario con token de 32 bytes hex y expiración de 1h.

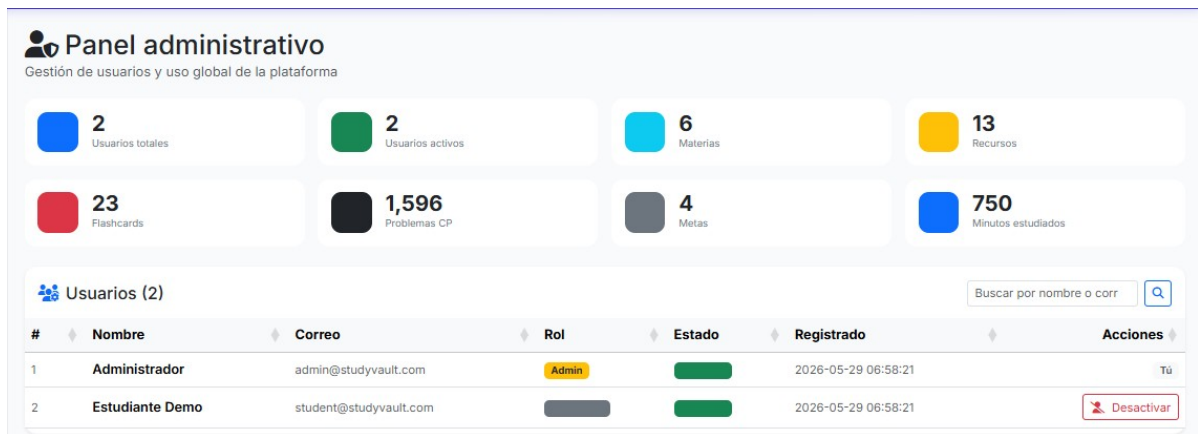


Figura 16: **Panel administrativo:** 8 tarjetas de totales globales + tabla paginada de usuarios con búsqueda y acción activar/desactivar.



Figura 17: **Rol Invitado:** sin iniciar sesión, accede al listado de plantillas públicas pero el botón cambia a “Crear cuenta para clonar”.

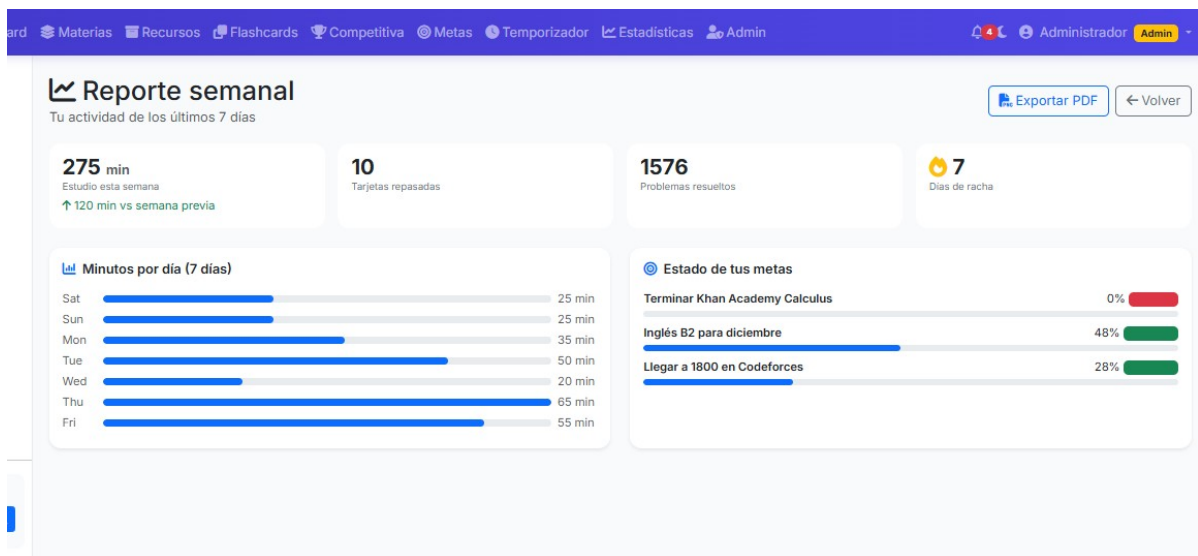


Figura 18: **Reporte semanal:** comparativa de minutos vs semana previa, tarjetas repasadas, problemas resueltos, racha y estado de metas. Botón “Exportar PDF” con CSS @media print.

8. Explicación técnica

8.1 Arquitectura general

StudyVault sigue el patrón **Modelo–Vista–Controlador** sin framework externo, implementado en **PHP 8 OOP** sobre **Apache + MySQL**.

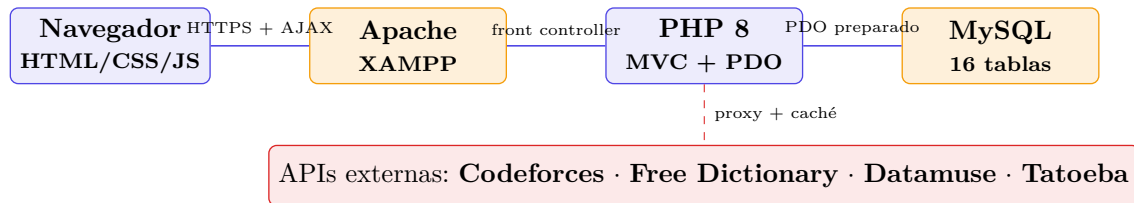


Figura 19: Pipeline general de una petición. Las APIs externas se consumen exclusivamente desde el servidor (proxy PHP) para evitar CORS, controlar el rate limit y cachear las respuestas.

8.2 Estructura de carpetas

```

studyvault/
config/      init.php  database.php  config.php
models/      User, Subject, Resource, Unit, Goal, Flashcard,
              CpProblem, CpTemplate, StudySession,
              PasswordReset, Notification, LoginThrottle
controllers/ AuthController, SubjectController, ResourceController,
              UnitController, GoalController, TimerController,
              FlashcardController, CpController, DashboardController,
              StatsController, NotificationController, AdminController,
              ReportController
views/       auth/ dashboard/ subjects/ resources/ units/
              flashcards/ cp/ goals/ timer/ report/
              stats/ notifications/ admin/ partials/
api/         dictionary.php datamuse.php tatoeba.php
              search.php resource_status.php
assets/      css/app.css js/app.js vendor/chart.umd.min.js
              uploads/ (con .htaccess)
tests/       sm2_test.php goal_test.php units_test.php
              auth_test.php security_test.php
studyvault.sql + migrations_v2..v8.sql
  
```

8.3 Flujo de una petición

1. El navegador envía una petición a `index.php?page=X&action=Y`.
2. `index.php` requiere `config/init.php`, que:
 - Carga `.env` en variables de entorno.
 - Configura el manejo de errores (`set_exception_handler` con `log`).
 - Registra el **autoloader** para `models/` y `controllers/`.
 - Inicia la sesión con cookies endurecidas.
 - Aplica las **cabeceras de seguridad** (incluida CSP).
 - Genera el **token CSRF** si no existe.
3. Si el método es POST, `index.php` llama a `csrf_verify()`. Si falta el token o no coincide, responde **HTTP 403** y termina.
4. El `match` sobre `?page=...&action=...` despacha al método del controlador correspondiente.

5. El controlador valida, llama al modelo (PDO preparado) y responde:

- Vista PHP (incluye `partials/header.php` + contenido + `partials/footer.php`).
- O `json_response(...)` para endpoints AJAX.

8.4 Algoritmo SM-2 (núcleo de retención)

Función pura, sin acceso a BD, ubicada en `models/Flashcard.php`. Reutilizada por `Flashcard::review()` y por `CpProblem::reviewProblem()`.

Listing 1: `Flashcard::sm2` (extracto)

```
public static function sm2(float $ef, int $reps, int $interval, int $quality):
    array {
        $quality = max(0, min(5, $quality));
        if ($quality < 3) {
            $reps = 0;
            $interval = 1;           // reset si fallaste
        } else {
            if ($reps === 0)        $interval = 1;
            elseif ($reps === 1)    $interval = 6;
            else                     $interval = (int) round($interval * $ef);
            $reps++;
        }
        $ef = $ef + (0.1 - (5 - $quality) * (0.08 + (5 - $quality) * 0.02));
        if ($ef < 1.3) $ef = 1.3;   // piso del ease factor
        $status = $interval >= 90 ? 'mastered'
            : ($interval >= 21 ? 'mature' : 'learning');
        return compact('ef', 'reps', 'interval', 'status');
    }
```

Mapecto de los 4 botones a la calidad SM-2:

Botón	Calidad	Efecto
Otra vez	2	Reinicia: <code>reps=0</code> , <code>interval=1</code> , debes verla mañana
Difficil	3	Sube intervalo pero con ease factor menor
Bien	4	Comportamiento típico de éxito
Fácil	5	Sube ease factor más rápido

8.5 Seguridad: defensa en profundidad

1. **Anti SQL Injection:** 100% PDO con *prepared statements*. Sin excepción.
2. **Anti CSRF:** token por sesión (32 bytes hex). Helper `csrf_field()` en cada `<form>`; en AJAX se envía como campo `csrf_token` o cabecera `X-CSRF-Token`. Verificación con `hash_equals()` (tiempo constante).
3. **Anti IDOR:** cada query lleva `WHERE user_id = ?`. Hay pruebas en `tests/security_test.php` que crean dos usuarios y verifican que A no puede leer ni modificar lo de B.
4. **Anti XSS:** `htmlspecialchars()` en toda salida. CSP restringe orígenes:

```
Content-Security-Policy: default-src 'self';
script-src 'self' 'unsafe-inline' cdn.jsdelivr.net cdnjs.cloudflare.com
code.jquery.com cdn.datatables.net;
style-src 'self' 'unsafe-inline' (mismos CDNs) fonts.googleapis.com;
img-src 'self' data:; media-src https:; connect-src 'self';
```

5. **Contraseñas:** `password_hash($pwd, PASSWORD_DEFAULT)` con `bcrypt`; comparación con `password_verify()`

6. **Sesiones:** `session_regenerate_id(true)` al autenticar; cookies `HttpOnly`, `SameSite=Lax`, `Secure` bajo HTTPS.
7. **Throttling de login:** `LoginThrottle` cuenta intentos por IP+email en la tabla `login_attempts`; tras 5 fallos en 15 min, bloquea con mensaje.
8. **Cabeceras de seguridad:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin`.
9. **Uploads:** `.htaccess` en `assets/uploads/` desactiva la ejecución de PHP en ese directorio (mitiga subidas maliciosas con doble extensión).
10. **Credenciales:** `.env` fuera del repositorio; `APP_DEBUG=false` en producción.

8.6 Integración con APIs externas

Todas las APIs se consumen vía **proxies PHP** (`api/*.php`) con tres ventajas:

1. **Evita CORS:** el navegador habla solo con `localhost/studyvault/api/`, no con dominios externos.
2. **Caché:** las respuestas de Free Dictionary y del problemset de Codeforces se guardan localmente.
3. **Rate limiting:** `rate_limit($key, $max, $seconds)` por sesión PHP.

API	Proxy local	Uso
Free Dictionary	<code>api/dictionary.php</code>	Definición + fonética + audio
Datamuse	<code>api/datamuse.php</code>	Colocaciones (palabras coexistentes)
Tatoeba	<code>api/tatoeba.php</code>	Frases de ejemplo reales
Codeforces	dentro de <code>CpController</code>	Rating + problemas + concursos + catálogo

8.7 Pruebas automatizadas

53 assertions en `tests/` ejecutables con `php tests/<archivo>.php`:

- `sm2_test.php` – 11 casos sobre el algoritmo SM-2 (primer/segundo/tercer acierto, fallo reinicia, piso ease factor, estados `mature/mastered`).
- `goal_test.php` – 10 casos sobre progreso ponderado y cálculo de pace (sin fecha, antes de la fecha, atrasado, completado, fecha vencida).
- `units_test.php` – 6 casos sobre % de unidades completadas (0 unidades, 0 de 5, 3 de 10, 5 de 5, redondeo).
- `auth_test.php` – CSRF, hashing, throttling.
- `security_test.php` – IDOR por dueño (un usuario no puede ver/modificar recursos de otro).

8.8 Métricas finales

Métrica	Valor
Tablas en la base de datos	16
Modelos PHP	12
Controladores	13
Vistas	26
Proxies de API externa	5
APIs externas integradas	4
Migraciones documentadas (v2–v8)	7
Pruebas automatizadas	53 assertions
Gráficas Chart.js	6
Requerimientos funcionales	32 (RF)
Requerimientos no funcionales	19 (RNF)
Casos de uso	19 (CU)

8.9 Conclusiones técnicas

StudyVault demuestra que es posible construir una aplicación web **integrada y útil** aplicando únicamente conceptos del curso, sin recurrir a frameworks. La separación MVC, el uso disciplinado de PDO preparado, la verificación con pruebas automáticas y la defensa en profundidad (CSRF + anti-IDOR + CSP + throttling) ofrecen una base sólida para evolucionar el sistema hacia un producto multiusuario.

El proyecto cumple las **10 especificaciones obligatorias** de la rúbrica y supera el mínimo en **seguridad, consumo de APIs** (4 en lugar de 1) y **funcionalidades opcionales** (tema oscuro, bitácora, recuperación de contraseña, notificaciones, exportación PDF, ordenamiento con DataTables, gráficas Chart.js).